On the Expressivity Role of LayerNorm in Transformers' Attention

Shaked Brody[†], **Uri Alon**[♠], **Eran Yahav**[†]
[†] Technion, Israel
[♠] Language Technologies Institute, Carnegie Mellon University, USA
{shakedbr,yahave}@cs.technion.ac.il
ualon@cs.cmu.edu

arXiv:2305.02582v2 [cs.LG] 11 May 2023

Abstract

Layer Normalization (LayerNorm) is an inherent component in all Transformer-based models. In this paper, we show that LayerNorm is crucial to the expressivity of the multi-head attention layer that follows it. This is in contrast to the common belief that LayerNorm's only role is to normalize the activations during the forward pass, and their gradients during the backward pass.

We consider a geometric interpretation of LayerNorm and show that it consists of two components: (a) *projection* of the input vectors to a $d-1$ space that is orthogonal to the $[1, 1, ..., 1]$ vector, and (b) *scaling* of all vectors to the same norm of $\sqrt{d}$. We show that each of these components is *important for the attention layer that follows it in Transformers*: (a) *projection* allows the attention mechanism to create an attention query that attends to all keys equally, offloading the need to learn this operation by the attention; and (b) *scaling* allows each key to potentially receive the highest attention, and prevents keys from being "un-select-able". We show empirically that Transformers do indeed benefit from these properties of LayeNorm in general language modeling and even in computing simple functions such as "majority". Our code is available at https://github.com/tech-srl/layer_norm_expressivity_role.

1 Introduction

LayerNorm (Ba et al., 2016) is the most commonly used normalization technique in modern neural networks such as Transformers (Vaswani et al., 2017).

Originally, Ba et al. (2016) motivated LayerNorm as an efficient way of normalizing the activations during the forward pass or providing distribution stability as in batch normalization (Ioffe and Szegedy, 2015). Later, Xu et al. (2019) and Xiong et al. (2020) argued that more importantly than normalizing forward activations, LayerNorm stabilizes the gradients during the *backward* pass.

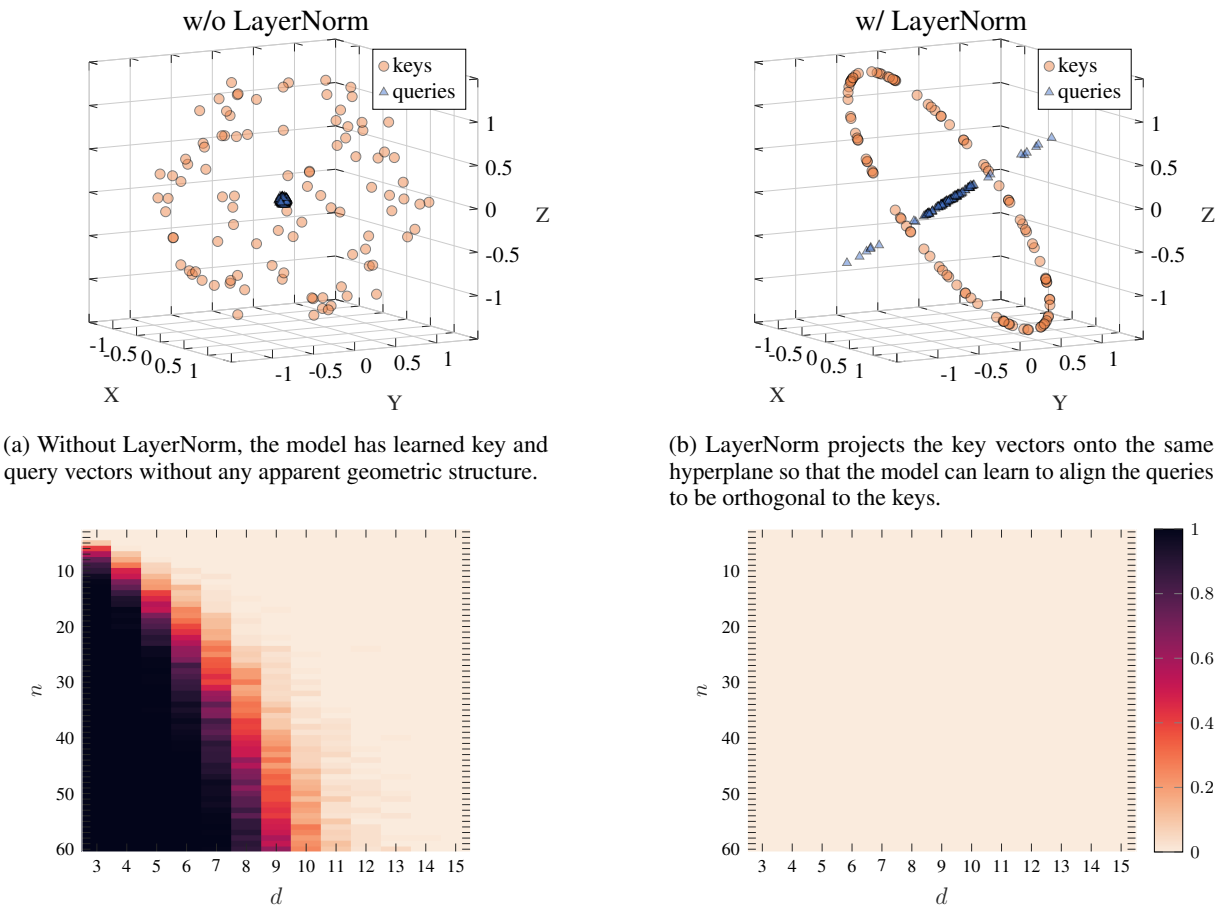
However, in this work, we show that LayerNorm, which was originally proposed for RNNs, has an additional crucial role in the theoretical and practical *expressivity* of the multi-head attention layer that follows it in Transformers. [1] That is, LayerNorm makes it easier for the Transformer to learn certain functions during training.

LayerNorm can be seen as two independent components: *projection* and *scaling*, that were merged into a single operator. First, LayerNorm *projects* its inputs onto a particular $d-1$ space that is orthogonal to the "ones" $\vec{\mathbb{1}} = [1, 1, ..., 1]$ vector. This allows the attention layer that follows the LayerNorm to create queries that are close to $\vec{\mathbb{1}}$, and thus attend to all keys equally, when needed, regardless of the identity of the keys. In Section 3 we show that this projection helps, for example, computing the "majority" among token types in a sequence.

Figure 1a shows how without LayerNorm, the keys and queries in a Transformer's attention have no apparent geometric structure. In contrast, Figure 1b shows that LayerNorm has projected all keys to the hyperplane that is orthogonal to the $\vec{\mathbb{1}}$ vector. Further, the attention mechanism has learned queries that are close to $\vec{\mathbb{1}}$, making them attend equally to any possible key, when trained to compute "majority". We analyze and prove this in Section 4.1.

The second component of LayerNorm is *scaling*: We show that LayerNorm scales the projected input to have an ℓ^2 norm of exactly $\sqrt{d}$. In Section 3, we show that scaling the input vectors prevents the problem of "unselectable" keys (Demeter et al., 2020; Grivas et al., 2022), where some key vectors are contained in the convex hull formed by the other keys, and thus can never get the highest attention score. Figures 1c and 1d show the average fraction

[1] Xiong et al. (2020) discuss the differences between placing LayerNorm *before* and *after* a Transformer layer. However, even when placing the LayerNorm *after* the layer, it appears right before the multi-head attention of the *next* layer.



(a) Without LayerNorm, the model has learned key and query vectors without any apparent geometric structure.

(b) LayerNorm projects the key vectors onto the same hyperplane so that the model can learn to align the queries to be orthogonal to the keys.

(c) Without LayerNorm, there are "unselectable" key vectors that cannot be selected by getting the maximal attention score (marked in darker colors).

(d) LayerNorm eliminates the problem of "unselectable" key vectors: Applying LayerNorm allows any key to get the highest attention score.

Figure 1: Figures 1a and 1b show the effect of *projection* in LayerNorm, which makes all keys lie on the hyperplane that is orthogonal to the $\vec{\mathbb{1}}$ vector. Figures 1c and 1d show the effect of *scaling*, where n is the number of vectors, d is the dimension, and the color represents the average fraction of "unselectable" key vectors.

(out of 100 runs) of "unselectable" vectors which were randomly drawn from the normal distribution. As shown in Figure 1c, without LayerNorm, the probability of getting "unselectable" keys can be very high in certain settings. Nonetheless, as shown in Figure 1d, with LayerNorm, every key vector is always selectable. We analyze and prove this in Section 4.2.

These results reveal new aspects of the commonly used LayerNorm and show its importance to the attention mechanism in Transformers.

2 Decomposing LayerNorm

Given an input $\boldsymbol{x} \in \mathbb{R}^d$, LayerNorm is defined as the following quotient:[2]

$$\boldsymbol{y} = \frac{\boldsymbol{x} - \boldsymbol{\mu}}{\sigma} \tag{1}$$

[2]Following Xu et al. (2019) and for simplicity, we drop the learned bias and gain terms.

where μ is the coordinate-wise average of $\boldsymbol{x}$ and σ is the coordinate-wise standard deviation:

$$\mu = \frac{1}{d}\sum_{i=1}^{d} x_i, \quad \sigma = \sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2} \tag{2}$$

$$\boldsymbol{\mu} = [\mu, \mu, ..., \mu] \in \mathbb{R}^d \tag{3}$$

We start with the numerator $\boldsymbol{x} - \boldsymbol{\mu}$ and show that it corresponds to the projection of $\boldsymbol{x}$ onto the hyperplane $\mathcal{H}$ defined by the normal vector $\vec{\mathbb{1}} = [1, 1, ..., 1] \in \mathbb{R}^d$:

$$\begin{aligned} (\boldsymbol{x} - \boldsymbol{\mu}) \cdot \vec{\mathbb{1}} &= \boldsymbol{x} \cdot \vec{\mathbb{1}} - \boldsymbol{\mu} \cdot \vec{\mathbb{1}} \\ \sum_{i=1}^{d} x_i &- \left(\frac{1}{d}\sum_{i=1}^{d} x_i\right) \cdot d = 0 \end{aligned} \tag{4}$$

That is, $\boldsymbol{x} - \boldsymbol{\mu}$ is always orthogonal to the $\vec{\mathbb{1}}$ vector. Next, we show that the denominator scales the

projected vector to have a norm of exactly $\sqrt{d}$:

$$
\begin{aligned}
||\boldsymbol{x}-\boldsymbol{\mu}|| &= \sqrt{\sum_{i=1}^{d}(x_i-\mu)^2} \\
&= \sqrt{d}\sqrt{\frac{1}{d}\sum_{i=1}^{d}(x_i-\mu)^2} = \sqrt{d}\cdot\sigma
\end{aligned}
\tag{5}
$$

Then, σ in the denominator of LayerNorm scales the projected vector to have a norm of exactly $\sqrt{d}$.

LayerNorm can thus be seen as two independent components: (a) *projection* of the input vectors onto the hyperplane orthogonal to $\vec{\mathbb{1}}$, and (b) *scaling* of the projected vectors to have a norm of $\sqrt{d}$.

3 Expressivity Role in Attention

Each of the components of LayerNorm supports the Transformer's attention in a different way. *Projection* helps creating a query that attends to all keys equally, when needed, while *scaling* helps the model to avoid the problem of "unselectable" keys.

Recall that in Transformers, given vectors $\boldsymbol{q}$ and $\boldsymbol{k}$, the attention scoring function is defined as:

$$
s(\boldsymbol{q},\boldsymbol{k}) = \frac{(\boldsymbol{qQ})(\boldsymbol{kK})^\top}{\sqrt{d}} = \left(\frac{\boldsymbol{qQK}^\top}{\sqrt{d}}\right)\boldsymbol{k}^\top \tag{6}
$$

From this point, we refer to $\left(\frac{\boldsymbol{qQK}^\top}{\sqrt{d}}\right)$ as "query", and to $\boldsymbol{k}$ as "key".

3.1 Projection

Projecting all attention keys to the same hyperplane can help attention attending to all keys equally. Since all projected keys are orthogonal to the hyperplane's normal $\vec{\mathbb{1}}$, this fact can be exploited in the training process by learning weights such that the queries will be parallel to $\vec{\mathbb{1}}$. That is, the attention can learn weights such that $\left(\frac{\boldsymbol{qQK}^\top}{\sqrt{d}}\right) \approx c\cdot\vec{\mathbb{1}}$, which will result in $\left(\frac{\boldsymbol{qQK}^\top}{\sqrt{d}}\right)\cdot\boldsymbol{k} \approx 0$ and thus $s(\boldsymbol{q},\boldsymbol{k}) = 0$ for any key $\boldsymbol{k}$.

Giving an equal score to all the keys can help, for example, in computing "majority", where the model needs to find the most frequent token in the input. In Section 4.1, we show that in the "majority" task, a Transformer learns to align the queries to be orthogonal to the keys, allowing a much faster convergence.

3.2 Scaling

Scaling the attention keys to the same size allows a Transformer to avoid the problem of "unselectable" keys, where there are keys to which the attention cannot focus and give the highest score.

Let $\mathcal{S} = \{\boldsymbol{h}_1, ..., \boldsymbol{h}_{n-1}, \boldsymbol{h}_n\}$ be a set of key vectors, such that $\boldsymbol{h}_n$ lies within the convex hull formed by the other vectors in $\mathcal{S}$. Due to the linearity of the attention scoring function s, the attention mechanism cannot select $\boldsymbol{h}_n$ by giving it the highest attention score. We formulate this in the following theorem:

Theorem 1. *Given a set of vectors $\mathcal{S} = \{\boldsymbol{h}_1, ..., \boldsymbol{h}_{n-1}, \boldsymbol{h}_n\}$ such that $\boldsymbol{h}_n$ is interior to the convex hull of $\mathcal{S}$, then for all $\boldsymbol{v} \in \mathbb{R}^d$ (s.t. $\boldsymbol{v} \neq \vec{\mathbf{0}}$):*

$$
\max_{i\in[n-1]} \boldsymbol{v}^\top\boldsymbol{h}_i > \boldsymbol{v}^\top\boldsymbol{h}_n
$$

This means that key vectors that are inside the convex hull cannot be selected by getting the highest attention score. Applying LayerNorm to the keys ensures that all keys are *scaled* to the same size, and thus none of them lies inside the convex hull of $\mathcal{S}$. This allows the attention mechanism to potentially focus and select any desired key. The proof of Theorem 1 is provided in Appendix A.

In Section 4.2 we show that this happens in practice when we train a Transformer on language modeling: There are key vectors that lie within the convex hull and therefore cannot be selected by receiving the maximal attention.

4 Experimental Results

In this section, we empirically show the effects of the LayerNorm components – *projection* and *scaling* – on the attention mechanism in Transformers. We first show how a Transformer learns to use the *projection* of LayerNorm to compute "majority"; then, we show that *scaling* allows the model to avoid the problem of "unselectable" keys, allowing the attention mechanism to focus on any key.

4.1 Computing Majority

We demonstrate the ability to compute "majority" using the *projection* property. In this task, the goal is to predict the majority token type in a sequence. Given a sequence of tokens $t_1, t_2, ..., t_n \in \{C_1, C_2, ..., C_k\}$, the goal is to predict the token type C_i that occurs the most among $t_1, t_2, ..., t_n$. For a, a, b, b, b, c, c, for example, the model is trained to predict the output b, b, b, b, b, b, b. Solving this task can be performed simply using exact averaging of the keys.

We trained a single-layer Transformer encoder with dimension $d = 8$ and a single attention head.